

Arquitetura do Projeto 05-nest-clean

Clean Architecture & DDD em NestJS — API de fórum Q&A (projeto Rocketseat Ignite)

Gerado em 02/07/2026

Visão geral

O projeto implementa **Clean Architecture** com **DDD tático**, em três camadas concêntricas. A regra de ouro: **as dependências sempre apontam para dentro**. A camada `domain` nunca sabe que Prisma, Redis ou NestJS existem.

```
infra → domain → core
(detalhes) (regras) (genérico)
```

1. core/ — o alicerce, sem regra de negócio

Either<L, R> — padrão Result

`src/core/either.ts`

Em vez de use cases lançarem `throw`, eles retornam um `Either`: ou um `Left` (erro) ou um `Right` (sucesso).

```
export class Left<L, R> { value: L; isLeft() { return true }; isRight() { return false } }
export class Right<L, R> { value: R; isLeft() { return false }; isRight() { return true } }
export type Either<L, R> = Left<L, R> | Right<L, R>
```

Vantagem: o *tipo* de retorno do use case já documenta todos os erros possíveis, e o TypeScript obriga a checar `isLeft()` antes de usar o valor — erro de negócio deixa de ser uma exceção invisível e vira um valor de retorno normal.

Entity e AggregateRoot

`src/core/entities/entity.ts`, `src/core/entities/aggregate-root.ts`

`Entity<Props>` é a base: tem `id` (um `UniqueEntityID`) e `props` protegidas. `AggregateRoot` estende `Entity` e adiciona uma lista de `_domainEvents` — é o marcador de "isso é a raiz de um agregado DDD, pode disparar eventos".

```
protected addDomainEvent(domainEvent: DomainEvent): void {
  this._domainEvents.push(domainEvent)
  DomainEvents.markAggregateForDispatch(this) // avisa o dispatcher global
}
```

DomainEvents — pub/sub estático em memória

src/core/events/domain-events.ts

Não é uma fila real, é só um `Record<string, callback[]>` estático na classe. Funciona assim:

1. Uma entidade chama `addDomainEvent(...)` → o agregado é "marcado" (`markAggregateForDispatch`).
2. Handlers se inscrevem via `DomainEvents.register(callback, "NomeDoEvento")`.
3. Quando o repositório efetivamente persiste no banco, ele chama `DomainEvents.dispatchEventsForAggregate(id)` — só **depois** disso os eventos disparam de verdade, e a lista é limpa.

Isso garante que eventos só disparam se a transação de escrita deu certo — sem risco de notificar algo que não foi salvo.

2. domain/ — as regras de negócio

Dividido em dois bounded contexts: `forum` (perguntas, respostas, comentários) e `notification`.

Entidades ricas

src/domain/forum/enterprise/entities/question.ts

Não são DTOs anêmicos — os setters carregam lógica de negócio:

```
set title(title: string) {
  this.props.title = title
  this.props.slug = Slug.createFromText(title) // recalcula o value object slug junto
  this.touch()
}

set bestAnswerId(bestAnswerId) {
  // ...
  if (/* mudou de fato */) {
    this.addDomainEvent(new QuestionBestAnswerChosenEvent(this, bestAnswerId))
  }
  this.props.bestAnswerId = bestAnswerId
  this.touch()
}
```

Escolher a melhor resposta de uma pergunta **automaticamente** dispara um evento de domínio, sem que o use case precise saber disso.

Use Cases

src/domain/forum/application/use-cases/answer-question.ts

Cada ação do sistema é uma classe `@Injectable()` com um único `execute()`. É um estilo CQRS informal — não usa `@nestjs/cqrs`, o controller chama o use case diretamente.

```
@Injectable()
export class AnswerQuestionUseCase {
  constructor(private answersRepository: AnswersRepository) {} // interface, não implementa

  async execute({ authorId, questionId, content, attachmentsIds }) {
    const answer = Answer.create({ content, authorId: ..., questionId: ... })
    // ...
    await this.answersRepository.create(answer)
    return right({ answer })
  }
}
```

O construtor injeta `AnswersRepository`, uma **classe abstrata** (interface) definida em `domain/.../repositories/answers-repository.ts` — o use case não sabe se por trás tem Prisma ou um mock em memória.

Subscribers

src/domain/notification/application/subscribers/on-answer-created.ts

Ligam os dois bounded contexts sem acoplamento direto: quando uma resposta é criada no `forum`, o `notification` reage e cria uma notificação, mas o `forum` nunca importa nada de `notification`.

```

@Injectables()
export class OnAnswerCreated implements EventHandler {
  constructor(
    private questionsRepository: QuestionsRepository,
    private sendNotification: SendNotificationUseCase,
  ) {
    this.setupSubscriptions()
  }

  setupSubscriptions(): void {
    DomainEvents.register(this.sendNewAnswerNotification.bind(this), AnswerCreatedEvent.name)
  }

  private async sendNewAnswerNotification({ answer }: AnswerCreatedEvent) {
    const question = await this.questionsRepository.findById(answer.questionId.toString())
    if (question) {
      await this.sendNotification.execute({ recipientId: question.authorId.toString(), /*
    }
  }
}

```

Esse subscriber é instanciado no bootstrap (`EventsModule`) só para forçar seu construtor a rodar e se registrar no dispatcher — um truque comum desse padrão.

3. infra/ — onde a realidade entra

Repositório Prisma

src/infra/database/prisma/repositories/prisma-questions-repository.ts

Implementa a interface `QuestionsRepository` do domain. É aqui que Prisma, cache Redis e disparo de eventos se juntam no mesmo método:

```

async save(question: Question): Promise<void> {
  const data = PrismaQuestionMapper.toPrisma(question) // Entity → shape do Prisma

  await Promise.all([
    this.prisma.question.update({ where: { id: question.id.toString() }, data }),
    this.questionAttachmentsRepository.createMany(question.attachments.getNewItems()),
    this.questionAttachmentsRepository.deleteMany(question.attachments.getRemovedItems()),
    this.cache.delete(`question:${data.slug}:details`), // invalida cache
  ])

  DomainEvents.dispatchEventsForAggregate(question.id) // só dispara evento após salvar
}

```

`findDetailsBySlug` usa cache-aside clássico: tenta o Redis primeiro; se não tiver, busca no Postgres e popula o cache com TTL de 15 min.

O `getNewItems()` / `getRemovedItems()` vêm do `WatchedList` — uma lista "observável" que compara o estado atual de anexos com o original e diz o que foi adicionado/removido, sem o repositório precisar fazer diff manual.

Inversão de dependência via módulos Nest

Em cada módulo de infra, a ligação interface → implementação é feita explicitamente:

```

// CacheModule
{ provide: CacheRepository, useClass: RedisCacheRepository }

```

Isso é o que permite trocar `RedisCacheRepository` por um fake em teste sem tocar em nenhuma linha de use case.

AppModule

src/infra/app.module.ts

É a composição raiz: junta `AuthModule`, `HttpModule`, `EnvModule` (valida `.env` com Zod), `EventsModule` (registra os subscribers). Simples de propósito — a complexidade real está distribuída nos módulos de cada camada.

Stack e infraestrutura

Área	Tecnologia
Framework	NestJS 10
ORM / Banco	Prisma 5.2 + PostgreSQL

Cache	Redis (ioredis), TTL de 15 min
Autenticação	JWT RS256 (@nestjs/jwt + passport-jwt)
Validação	Zod (HTTP e env)
Upload	@aws-sdk/client-s3 (Cloudflare R2)
Testes	Vitest + @faker-js/faker, repositórios in-memory
Infra local	docker-compose (Postgres + Redis, sem Dockerfile da app)
CI	Não configurado (sem .github/)

Por que essa arquitetura existe

1. **Testabilidade:** como use cases dependem só de interfaces, os testes unitários rodam 100% em memória (repositórios fake), sem subir Postgres nem Redis — rápido e determinístico.
2. **Independência de framework:** `domain` não importa nada do NestJS (exceto `@Injectable()`, que é só metadata). Trocar Nest por Express puro amanhã não afeta o domain.
3. **Regra de negócio centralizada na entidade**, não espalhada em use cases/controllers — evita duplicação de validação e side-effects.
4. **Eventos desacoplam contextos** (`forum` não conhece `notification`) sem precisar de fila de mensageria real — troca simplicidade por não ter garantias de entrega/retry (aceitável no escopo do curso, mas ponto de atenção numa arquitetura de produção real).